

How is Flash Attention 2 different from Flash Attention?

- Long context: GPT-4 (32k), MosaicML's MPT (65k), Anthropic's Claude (100k).
- Attention layer is the main bottleneck in scaling to longer sequences, as its runtime and memory increase quadratically in the sequence length.
- FlashAttention: IO-aware exact attention algo that exploits the asymmetric GPU memory hierarchy (SRAM, HBM) to bring significant memory saving and runtime speedup.
- But FlashAttention is still not as fast as optimized matrix-multiply (GEMM).
 - Forward pass is ~30-50% of max FLOPs/s of A100.
 - Backward pass is ~25-35% of max FLOPs/s
 - Optimized GEMM is ~80-90% of max FLOPs/s
- FlashAttention 2
 - 2× speedup compared to FlashAttention
 - 50-73% of max FLOPs/s on A100 (73% forward pass; 63% backward pass)
- GPU execution model
 - Massive #threads to execute an operation (kernel).
 - Thread blocks run on streaming multiprocs (A100 has 108).
 - Thread block contains warps (group of 32 threads).
 - Threads within a warp communicate by fast shuffle instructions or cooperate to perform matrix multiply. Warps within a thread block can communicate by reading from / writing to shared memory.
 - Each kernel loads inputs from HBM to registers and SRAM, computes, then writes outputs to HBM.
- What changed compared to FlashAttention?
 - Reduced non-matmul FLOPs
 - matmul throughput can be up to 16× higher than non-matmul throughput
 - Parallelized attention computation
 - Better work partitioning between warps

Dao, Tri. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." arXiv preprint arXiv:2307.08691 (2023).

Reducing non-matmul FLOPs

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^T \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}$$

- Consider 2 blocks in $\mathbf{S}$.
- Standard softmax

$$m = \max(\text{rowmax}(\mathbf{S}^{(1)}), \text{rowmax}(\mathbf{S}^{(2)})) \in \mathbb{R}^{B_r}$$

$$\ell = \text{rowsum}(e^{\mathbf{S}^{(1)} - m}) + \text{rowsum}(e^{\mathbf{S}^{(2)} - m}) \in \mathbb{R}^{B_r}$$

$$\mathbf{P} = [\mathbf{P}^{(1)} \quad \mathbf{P}^{(2)}] = \text{diag}(\ell)^{-1} \begin{bmatrix} e^{\mathbf{S}^{(1)} - m} & e^{\mathbf{S}^{(2)} - m} \end{bmatrix} \in \mathbb{R}^{B_r \times 2B_c}$$

$$\mathbf{O} = [\mathbf{P}^{(1)} \quad \mathbf{P}^{(2)}] \begin{bmatrix} \mathbf{V}^{(1)} \\ \mathbf{V}^{(2)} \end{bmatrix} = \text{diag}(\ell)^{-1} [e^{\mathbf{S}^{(1)} - m} \mathbf{V}^{(1)} + e^{\mathbf{S}^{(2)} - m} \mathbf{V}^{(2)}] \in \mathbb{R}^{B_r \times d}$$

- Flash Attention
 - Online softmax computes “local” softmax wrt each block and rescale to get the right output at the end.
- Flash Attention 2
 - We do not have to rescale both terms of the output update.
 - We can instead maintain an “un-scaled” version of output and keep around the statistics ℓ .
 - Only at the very end of the loop do we scale the final $\tilde{\mathbf{O}}(\text{last})$ by $\text{diag}(\ell^{(\text{last})})^{-1}$ to get the right output

$$m^{(1)} = \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r}$$

$$\ell^{(1)} = \text{rowsum}(e^{\mathbf{S}^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_r}$$

$$\tilde{\mathbf{P}}^{(1)} = \text{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)} - m^{(1)}} \in \mathbb{R}^{B_r \times B_c}$$

$$\mathbf{O}^{(1)} = \tilde{\mathbf{P}}^{(1)} \mathbf{V}^{(1)} = \text{diag}(\ell^{(1)})^{-1} e^{\mathbf{S}^{(1)} - m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d}$$

$$m^{(2)} = \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m$$

$$\ell^{(2)} = e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)} - m}) + \text{rowsum}(e^{\mathbf{S}^{(2)} - m}) = \ell$$

$$\tilde{\mathbf{P}}^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m^{(2)}}$$

$$\mathbf{O}^{(2)} = \text{diag}(\ell^{(1)} / \ell^{(2)})^{-1} \mathbf{O}^{(1)} + \tilde{\mathbf{P}}^{(2)} \mathbf{V}^{(2)} = \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(1)} - m} \mathbf{V}^{(1)} + \text{diag}(\ell^{(2)})^{-1} e^{\mathbf{S}^{(2)} - m} \mathbf{V}^{(2)} = \mathbf{O}$$



$$m^{(1)} = \text{rowmax}(\mathbf{S}^{(1)}) \in \mathbb{R}^{B_r}$$

$$\ell^{(1)} = \text{rowsum}(e^{\mathbf{S}^{(1)} - m^{(1)}}) \in \mathbb{R}^{B_r}$$

$$\tilde{\mathbf{O}}^{(1)} = e^{\mathbf{S}^{(1)} - m^{(1)}} \mathbf{V}^{(1)} \in \mathbb{R}^{B_r \times d}$$

$$m^{(2)} = \max(m^{(1)}, \text{rowmax}(\mathbf{S}^{(2)})) = m$$

$$\ell^{(2)} = e^{m^{(1)} - m^{(2)}} \ell^{(1)} + \text{rowsum}(e^{\mathbf{S}^{(2)} - m^{(2)}}) = \text{rowsum}(e^{\mathbf{S}^{(1)} - m}) + \text{rowsum}(e^{\mathbf{S}^{(2)} - m}) = \ell$$

$$\tilde{\mathbf{O}}^{(2)} = \text{diag}(e^{m^{(1)} - m^{(2)}}) \tilde{\mathbf{O}}^{(1)} + e^{\mathbf{S}^{(2)} - m^{(2)}} \mathbf{V}^{(2)} = e^{\mathbf{S}^{(1)} - m} \mathbf{V}^{(1)} + e^{\mathbf{S}^{(2)} - m} \mathbf{V}^{(2)}$$

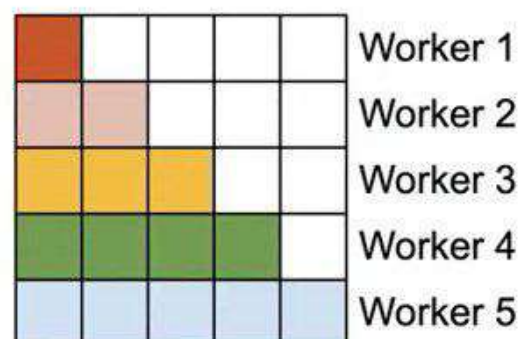
$$\mathbf{O}^{(2)} = \text{diag}(\ell^{(2)})^{-1} \tilde{\mathbf{O}}^{(2)} = \mathbf{O}.$$

Dao, Tri. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." arXiv preprint arXiv:2307.08691 (2023).

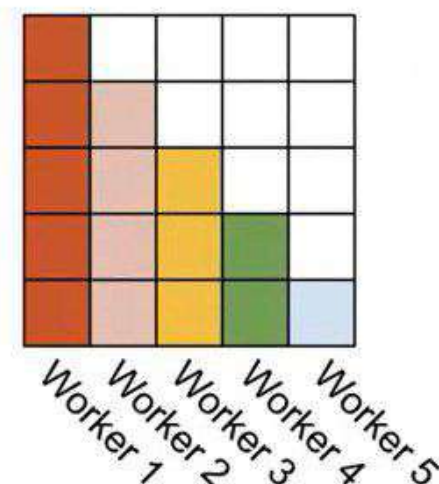
Parallelizing attention computation

- FlashAttention parallelizes over batch size (BS) and #heads.
- Thus, there are $BS \times \text{\#heads}$ thread blocks, each running on a streaming multiprocessor (SM).
- A100 -- 108 SMs.
- For long sequences (small BS or small #heads), to make better use of the SMs on GPU, need to also parallelize over sequence length.
- Flash Attention
 - For j-th (K,V), for i-th Q, Compute using K_j, V_j, Q_i in SRAM, and update O_i, l_i, m_i in HBM.
- Flash Attention 2
 - Swap order of loop
 - Parallelize outer loop (i.e., over sequence length dimension).
 - Leads to improved occupancy (fraction of GPU resources being used)
 - Also applicable to backward pass

Forward pass



Backward pass



Forward pass: each worker handles a block of rows of attention matrix.

Backward pass: each worker handles a block of columns.

Dao, Tri. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." arXiv preprint arXiv:2307.08691 (2023).

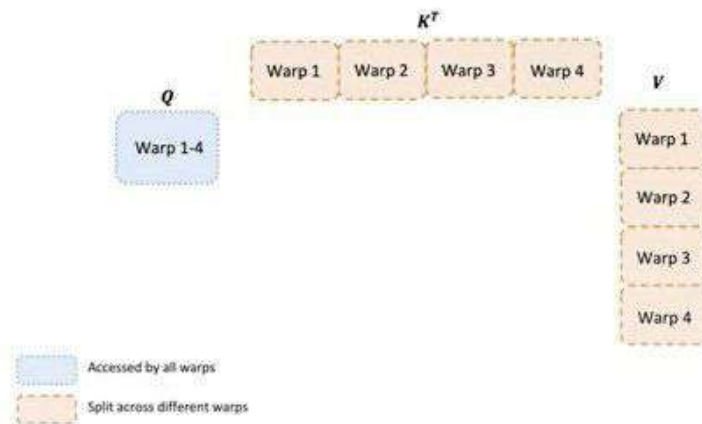
26-Jul-23

manishg.iitb@gmail.com

4

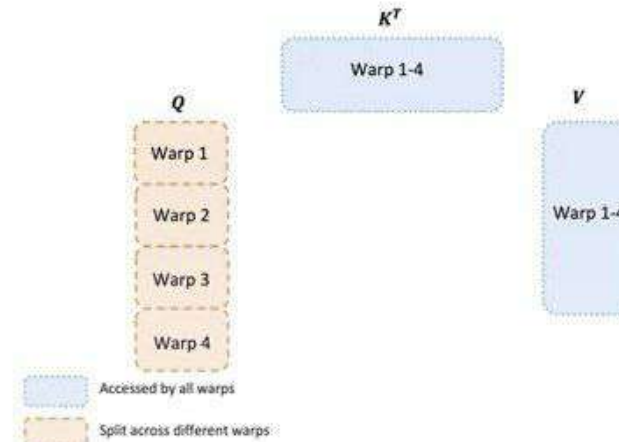
Work Partitioning Between Warps

- We typically use 4 or 8 warps per thread block.
- FlashAttention
 - Splits K and V across 4 warps while keeping Q accessible by all warps.
 - “split-K”: Each warp multiplies to get a slice of QK^T , then they need to multiply with a slice of V and communicate to add up the result.
 - Inefficient since all warps need to write their intermediate results out to shared memory, synchronize, then add up the intermediate results.



(a) FLASHATTENTION

- FlashAttention-2
 - Split Q across 4 warps while keeping K and V accessible by all warps.
 - After each warp computes a slice of QK^T , they just need to multiply with their shared slice of V to get their corresponding slice of the output.
 - No need for communication between warps.
 - Reduction in shared memory reads/writes yields speedup.
- Similar method applies to Backward pass also.



(b) FLASHATTENTION-2

Dao, Tri. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." arXiv preprint arXiv:2307.08691 (2023).

Results

- Benchmarking attention

- FlashAttention-2

- 1.7-3.0× faster than FlashAttention
 - 1.3-2.5× faster than FlashAttention in Triton
 - 3-10× faster than a standard PyTorch attention implementation.

- FlashAttention-2 reaches up to 230 TFLOPs/s, 73% of the theoretical max TFLOPs/s on A100 GPUs.

- End-to-end training speed

- Train GPT-style models of size 1.3B and 2.7B on sequence lengths either 2k or 8k
 - FlashAttention-2 yields up to 1.3× speedup compared to FlashAttention.
 - 2.8× speedup compared to a baseline without FlashAttention.
 - FlashAttention-2 reaches up to 225 TFLOPs/s (72% model FLOPs utilization) per A100 GPU.

Model	Without FLASHATTENTION	FLASHATTENTION	FLASHATTENTION-2
GPT3-1.3B 2k context	142 TFLOPs/s	189 TFLOPs/s	196 TFLOPs/s
GPT3-1.3B 8k context	72 TFLOPs/s	170 TFLOPs/s	220 TFLOPs/s
GPT3-2.7B 2k context	149 TFLOPs/s	189 TFLOPs/s	205 TFLOPs/s
GPT3-2.7B 8k context	80 TFLOPs/s	175 TFLOPs/s	225 TFLOPs/s

Dao, Tri. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." arXiv preprint arXiv:2307.08691 (2023).

26-Jul-23

manishg.iitb@gmail.com

Summary

- FlashAttention-2 is 2× faster than FlashAttention.
- FlashAttention-2 will also speed up training, finetuning, and inference of existing models.
- <https://github.com/Dao-AI-Lab/flash-attention>
- Thanks for watching!
- LinkedIn: <http://aka.ms/manishgupta>
- HomePage: <https://sites.google.com/view/manishg/>